

Table of Contents

Process Information	3
The C Standard Library	4
Static vs Dynamic Linking	4
Inspecting Executables with readelf	5
Tracing Dependencies with ldd	6
Knowledge Check Answers	7

Table of Figures

Figure 1: Code of procinfo.c	3
Figure 2: Compilation and execution of procinfo.c	3
Figure 3: Locations of C library headers and libc files	4
Figure 4: Static compilation of procinfo.c (gcc -static)	4
Figure 5: Dynamic compilation of procinfo.c (default gcc)	4
Figure 6: ELF Header of procinfo_dynamic (readelf -h)	5
Figure 7: Program Headers of procinfo_dynamic (readelf -l)	5
Figure 8: Dynamic Section of procinfo_dynamic (readelf -d)	6
Figure 9: Shared library dependencies of procinfo_dynamic (ldd)	6
Figure 10: ldd on the statically linked procinfo_static	6

Process Information

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
int main(void) {
    // Get current process ID
    pid_t my_pid = getpid();
    pid_t my_ppid = getppid();
    // Get current time
    time_t current_time = time(NULL);
    struct tm *time_info =
    localtime(&current_time);
    // Format and display information
    printf("=== Process Information
    ===\n");
    printf("My Process ID (PID): %d\n",
    my_pid);
    printf("Parent Process ID: %d\n",
    my_ppid);
    printf("Current Time: %s",
    asctime(time_info));
    printf("Executable Path:
    /proc/self/exe\n");
    return 0;
}
```

Figure 1: Code of procinfo.c

```
kritika@kritika-VirtualBox:~/4th/lab2.1$ nano procinfo.c
kritika@kritika-VirtualBox:~/4th/lab2.1$ nano procinfo.c
kritika@kritika-VirtualBox:~/4th/lab2.1$ gcc procinfo.c -o procinfo
kritika@kritika-VirtualBox:~/4th/lab2.1$ ./procinfo
=== Process Information===
My Process ID (PID): 10053
Parent Process ID: 3893
Current Time: Sun Oct  4 13:12:29 2026
Executable Path: /proc/self/exe
kritika@kritika-VirtualBox:~/4th/lab2.1$ s
```

Figure 2: Compilation and execution of procinfo.c

Summary

The C program procinfo.c was written using nano and includes the stdio.h, stdlib.h, unistd.h and time.h headers. It calls getpid() and getppid() to read the process IDs, and time() and localtime() to get the current time. The program was compiled with gcc procinfo.c -o procinfo and executed with ./procinfo. The output displayed the PID (10053), the parent PID (3893), the current date and time, and the executable path /proc/self/exe.

The C Standard Library

```
kritika@kritika-VirtualBox:~/4th/lab2.1$ ls /usr/include/stdio.h /usr/include/unistd.h
/usr/include/stdio.h /usr/include/unistd.h
kritika@kritika-VirtualBox:~/4th/lab2.1$ grep -w "printf" /usr/include/stdio.h | head -
n 22
/* Maximum length of printf output for a NaN. */
extern int printf (const char *__restrict __format, ...);
kritika@kritika-VirtualBox:~/4th/lab2.1$ grep -w "getpid" /usr/include/unistd.h | head
-n 2
extern __pid_t getpid (void) __THROW;
kritika@kritika-VirtualBox:~/4th/lab2.1$ ls /lib/x86_64-linux-gnu/libc.so.6
/lib/x86_64-linux-gnu/libc.so.6
kritika@kritika-VirtualBox:~/4th/lab2.1$ ls /usr/lib/x86_64-linux-gnu/libc.a
/usr/lib/x86_64-linux-gnu/libc.a
```

Figure 3: Locations of C library headers and libc files

Summary

The C Standard Library has two parts. The header files `stdio.h` and `unistd.h` are in `/usr/include/` and only contain declarations, such as the prototypes of `printf()` and `getpid()`, which was confirmed using `grep`. The compiled machine code is in library files: the shared library `libc.so.6` in `/lib/x86_64-linux-gnu/` and the static library `libc.a` in `/usr/lib/x86_64-linux-gnu/`.

Static vs Dynamic Linking

```
kritika@kritika-VirtualBox:~/4th/lab2.1$ gcc -static procinfo.c -o procinfo_static
kritika@kritika-VirtualBox:~/4th/lab2.1$ ls -lh procinfo_static
-rwxrwxr-x 1 kritika kritika 961K Oct  4 13:27 procinfo_static
kritika@kritika-VirtualBox:~/4th/lab2.1$ ls -lh procinfo_static
```

Figure 4: Static compilation of `procinfo.c` (`gcc -static`)

```
kritika@kritika-VirtualBox:~/4th/lab2.1$ gcc procinfo.c -o procinfo_dynamic
kritika@kritika-VirtualBox:~/4th/lab2.1$ ls -lh procinfo_dynamic
-rwxrwxr-x 1 kritika kritika 16K Oct  4 13:31 procinfo_dynamic
```

Figure 5: Dynamic compilation of `procinfo.c` (default `gcc`)

Summary

Linking resolves function names such as `printf` to actual code. The program was compiled statically using `gcc -static` and dynamically using the default `gcc` command. The static executable `procinfo_static` is 961K because the linker copied the needed `libc` code into it, while the dynamic executable `procinfo_dynamic` is only 16K because it just keeps a reference to the shared library `libc.so.6`, which is loaded at runtime.

Inspecting Executables with readelf

```
kritika@kritika-VirtualBox:~/4th/lab2.1$ readelf -h procinfo_dynamic
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                                ELF64
  Data:                                      2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  DYN (Position-Independent Executable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x1140
  Start of program headers:              64 (bytes into file)
  Start of section headers:              14264 (bytes into file)
  Flags:                                 0x0
  Size of this header:                   64 (bytes)
  Size of program headers:               56 (bytes)
  Number of program headers:             14
  Size of section headers:               64 (bytes)
  Number of section headers:             31
  Section header string table index:    30
```

Figure 6: ELF Header of procinfo_dynamic (readelf -h)

```
kritika@kritika-VirtualBox:~/4th/lab2.1$ readelf -l procinfo_dynamic

Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1140
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
are 14 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz            MemSiz             Flags             Align
PHDR             0x0000000000000040 0x0000000000000040 0x0000000000000040
                 0x0000000000000310 0x0000000000000310  R                 0x8
INTERP           0x0000000000000374 0x0000000000000374 0x0000000000000374
                 0x000000000000001c 0x000000000000001c  R                 0x1
LOAD             0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x00000000000007b8 0x00000000000007b8  R                 0x1000
LOAD             0x0000000000000100 0x0000000000000100 0x0000000000000100
                 0x0000000000000311 0x0000000000000311  R E                0x1000
LOAD             0x0000000000000200 0x0000000000000200 0x0000000000000200
                 0x00000000000001c0 0x00000000000001c0  R                 0x1000
LOAD             0x0000000000000d80 0x0000000000000d80 0x0000000000000d80
                 0x0000000000000290 0x0000000000000290  RW                0x1000
DYNAMIC          0x0000000000000d90 0x0000000000000d90 0x0000000000000d90
                 0x00000000000001f0 0x00000000000001f0  RW                 0x8
NOTE            0x0000000000000350 0x0000000000000350 0x0000000000000350
                 0x0000000000000024 0x0000000000000024  R                  0x4
NOTE            0x0000000000000217 0x0000000000000217 0x0000000000000217
                 0x0000000000000030 0x0000000000000030  R                 0x8
NOTE            0x000000000000021a 0x000000000000021a 0x000000000000021a
```

Figure 7: Program Headers of procinfo_dynamic (readelf -l)

```
kritika@kritika-VirtualBox:~/4th/lab2.1$ readelf -d procinfo_dynamic

Dynamic section at offset 0x2d90 contains 27 entries:
  Tag                Type                Name/Value
0x0000000000000001 (NEEDED)             Shared library: [libc.so.6]
0x000000000000000c (INIT)               0x1000
0x000000000000000d (FINI)               0x1304
0x0000000000000019 (INIT_ARRAY)         0x3d80
0x000000000000001b (INIT_ARRAYSZ)      8 (bytes)
0x000000000000001a (FINI_ARRAY)         0x3d88
0x000000000000001c (FINI_ARRAYSZ)      8 (bytes)
0x000000006ffffef5 (GNU_HASH)           0x390
0x0000000000000005 (STRTAB)           0x508
0x0000000000000006 (SYMTAB)           0x3b8
0x000000000000000a (STRSZ)            208 (bytes)
0x000000000000000b (SYMENT)           24 (bytes)
0x0000000000000015 (DEBUG)            0x0
0x0000000000000003 (PLTGOT)           0x3f80
0x0000000000000002 (PLTRELSZ)         192 (bytes)
0x0000000000000014 (PLTREL)            RELA
0x0000000000000017 (JMPREL)           0x6f8
0x0000000000000007 (RELA)             0x638
0x0000000000000008 (RELASZ)            192 (bytes)
0x0000000000000009 (RELAENT)           24 (bytes)
0x000000000000001e (FLAGS)             BIND_NOW
0x000000006fffffb (FLAGS_1)           Flags: NOW PIE
0x000000006fffffe (VERNEED)           0x5f8
0x000000006ffffff (VERNEEDNUM)        1
0x000000006fffff0 (VERSYM)            0x5d8
0x000000006fffff9 (RELACOUNT)          3
0x0000000000000000 (NULL)             0x0
kritika@kritika-VirtualBox:~/4th/lab2.1$
```

Figure 8: Dynamic Section of `procinfo_dynamic` (`readelf -d`)

Summary

The ELF Header (`readelf -h`) shows that the file is ELF64, little endian, of type DYN (Position-Independent Executable) for the x86-64 machine, with an entry point at 0x1140. The Program Headers (`readelf -l`) show the INTERP entry requesting the dynamic linker `/lib64/ld-linux-x86-64.so.2`, and the LOAD segments with R, R E (code) and RW (data) permissions. The Dynamic Section (`readelf -d`) lists the NEEDED shared library `libc.so.6` along with the BIND_NOW and PIE flags.

Tracing Dependencies with ldd

```
kritika@kritika-VirtualBox:~/4th/lab2.1$ ldd procinfo_dynamic
linux-vdso.so.1 (0x00007976bc462000)
libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x00007976bc200000)
/lib64/ld-linux-x86-64.so.2 (0x00007976bc464000)
```

Figure 9: Shared library dependencies of `procinfo_dynamic` (`ldd`)

```
kritika@kritika-VirtualBox:~/4th/lab2.1$ ldd procinfo_static
not a dynamic executable
```

Figure 10: `ldd` on the statically linked `procinfo_static`

Summary

The `ldd` command resolves each required library to its full path. For `procinfo_dynamic` it lists `linux-vdso.so.1` (provided by the kernel), `libc.so.6` found in `/usr/lib/x86_64-linux-gnu/`, and the dynamic linker `/lib64/ld-linux-x86-64.so.2`. For `procinfo_static`, `ldd` prints "not a dynamic executable", which confirms that the static binary has no runtime library dependencies.

Knowledge Check Answers

1. What is the fundamental difference between a C header file and a shared library?

A header file (.h) is a text file that contains only declarations such as function prototypes, macros, typedefs and structures. It tells the compiler what functions exist and how to call them, but it has no machine code. It is used at compile time and is stored in /usr/include/. A shared library (.so) is a precompiled binary that contains the actual machine code of the functions, for example printf() and getpid(). It is loaded into memory at runtime by the dynamic linker and one copy is shared by many programs. It is stored in /lib/x86_64-linux-gnu/.

2. What is static and dynamic linking?

Linking is the process of resolving function names, such as printf, to the actual code and addresses.

Static linking: the linker copies the required code from the static library (.a) directly into the executable at compile time (gcc -static procinfo.c -o procinfo_static). The executable is self-contained and does not need any .so file to run, but it is much larger (961 KB in this lab) and must be recompiled to get library updates.

Dynamic linking: the executable only stores references to shared libraries (.so), and the dynamic linker (/lib64/ld-linux-x86-64.so.2) loads them at runtime (gcc procinfo.c -o procinfo_dynamic). The executable is small (16 KB in this lab), the library code is shared in memory, and updating libc benefits every program. However, the program will not start if a required .so file is missing. This is the default on modern Linux.

3. What is the ELF Header? Write down the commands to check headers.

ELF (Executable and Linkable Format) is the binary file format used by executables on Linux. The ELF Header is the first part of an ELF file. It stores global information about the file: the magic number (7f 45 4c 46), the class (32-bit or 64-bit), the data encoding (endianness), the file type (EXEC or DYN), the target machine (for example x86-64) and the entry point address. The OS uses it to check that the file is a valid executable for the CPU.

Commands to inspect an ELF executable:

```
readelf -h procinfo_dynamic    # ELF header
readelf -l procinfo_dynamic    # program headers
readelf -d procinfo_dynamic    # dynamic section
ldd procinfo_dynamic           # shared library paths
```

4. What does bookmark mean? Provide an example.

A bookmark is a saved marker that points to a specific location, so the user can return to it quickly without searching again. For example, in Microsoft Word a bookmark can be inserted at a heading (Insert > Bookmark) and then used to jump directly to that place or to create a link to it. Similarly, a web browser bookmark saves the address of a web page, and in a Linux terminal a shortcut can be created with alias lab2='cd ~/4th/lab2.1' to return to the lab folder quickly.